

OpenAI Codex 最佳实践指南：提示工程、工具配置与安全策略

👤 Guide 📖 AI 编程实战 📄 约 4945 字 ⌚ 大约 16 分钟

大家好，我是 Guide。前面聊了 [Claude Code 的使用技巧](#)，这篇来看看 OpenAI 阵营的主力编程工具——**Codex**。

OpenAI 在 2025 年推出了 Codex 系列产品线，涵盖基于 o3 模型的云端软件工程智能体（codex-1）和开源的终端编码助手 Codex CLI。它和传统的代码补全不同，能自己读代码、跑测试、提 PR，完成从理解到交付的完整闭环。但想让它真正好用，提示工程、工具配置、安全策略这几环缺一不可。

这篇文章综合 OpenAI 官方博客、Codex CLI 开源仓库 README、官方提示工程指南等多个来源，整理成一份实践指南。通过本文你将搞懂：

1. 🌟 **Codex 云端智能体和 CLI 的定位差异**：各适合什么场景
2. 🌟 **提示工程的核心原则**：行动优先、上下文收集、代码质量标准
3. 🌟 **AGENTS.md 的分层机制**：怎么组织项目级指令
4. **安全模型的三级审批**：从建议到全自动的安全边界
5. **GPT-5.3 Codex API 的高级特性**：上下文压缩、Phase 机制、推理强度

一、认识 Codex：两条产品线与一个核心理念

Codex 云端智能体（codex-1）

OpenAI 发布了基于 o3 模型微调的 codex-1 云端智能体。它运行在 OpenAI 的安全沙箱中，可以读写代码、运行测试和命令行工具，甚至直接提交 Pull Request。三个核心特性：

- **自主执行**：你给出任务描述，它自行收集上下文、编写代码、运行测试，全程无需人工逐步引导
- **安全沙箱**：每个任务在独立的容器环境中运行，没有网络访问权限，防止对生产环境造成影响
- **AGENTS.md 指令机制**：类似于 `.cursorrules` 或 `CLAUDE.md`，你可以在仓库中放置 AGENTS.md 文件来定义项目级别的编码规范和约束

Codex 云端智能体目前通过 ChatGPT Pro、Business 和 Enterprise 计划提供访问，Plus 计划也于 2025 年 6 月起陆续开放。它支持两种工作模式：交互式对话和后台任务。后台模式下，你可以同时派发多个任务，每个任务在独立容器中并行执行。

一句话区分：云端智能体适合“挂后台跑大任务”，CLI 适合“坐电脑前盯着改代码”。两者定位不同，核心理念一致——长期自主、减少人工干预、以可交付的代码为目标。

Codex CLI：开源终端编码助手

Codex CLI 是一个完全开源的终端工具，用 Rust 编写，可以在本地机器上执行代码修改和 shell 命令。跟云端智能体的区别主要在运行环境和安全模型上：

维度	Codex 云端智能体	Codex CLI
运行环境	OpenAI 云端沙箱	本地机器
网络访问	无（隔离环境）	取决于本地权限
代码访问	GitHub 仓库集成	本地文件系统
安全模型	平台托管	三级审批模式
开源状态	闭源	完全开源（Rust）
适用计划	Pro/Business/Enterprise/Plus	Plus/Pro/Business/Edu/Enterprise

拓展一下：Codex CLI 默认使用的模型是 `codex-mini-latest`（基于 o4-mini），面向低延迟的代码问答和编辑场景优化。而云端智能体使用的是 `codex-1`（基于 o3），面向需要深度推理的复杂工程任务。两者的定位差异类似“轻量级助手”和“高级工程师”的区别。

二、提示工程：让 Codex 高效工作的核心

搞清楚了 Codex 两条产品线的区别，接下来是最关键的部分——怎么写好提示词。这部分的内容同时适用于云端智能体和 CLI。

★ 行动优先原则

这是 Codex 提示设计的第一原则——“**行动偏向**”（**Action Bias**）。好的提示应该引导模型直接交付可工作的代码，而不是用一堆问题结束回复。具体来说：

- 明确告知模型“交付可工作的代码，而不仅仅是计划”
- 模型应该默认做出合理假设并向前推进

- 只有在真正被阻塞（缺少关键信息或存在矛盾约束）时才向用户提问

反面示例：提示中要求模型“先列出计划，等确认后再执行”。这会让模型在完成工作前就停下来等待，严重降低效率。

正面示例：提示中写明“接到任务后立即开始工作，合理假设模糊部分，完成后展示结果。如有无法自行判断的阻塞问题，再询问用户。”

工程提示：官方提示词中有一段很关键——“每次推出都应以具体编辑或明确的阻塞者加上有针对性的问题结束”。这句话直接告诉模型：不要用“我来帮你分析一下”之类的废话收尾，要么给出代码改动，要么给出阻塞原因和具体问题。

★ 上下文收集策略

Codex 在开始修改代码之前，应该先充分理解代码库——这一点听起来理所当然，但实践中经常被忽略。提示中应明确要求：

1. **批量读取：**在调用工具前先想清楚需要哪些文件，然后一次性并行读取
2. **避免串行探索：**不要一个文件一个文件地逐个查看
3. **先搜索后新增：**在添加新实现之前，先搜索代码库中是否已有类似功能

这种“先规划、再并行”的策略可以显著减少往返轮次。

★ 代码质量标准

Codex 的定位是“有判断力的高级工程师”。在提示中应体现以下工程标准：

- 正确性优先于速度，避免冒险的捷径、投机性改动和拼凑式修复
- 遵循代码库现有约定，偏离时需要说明理由
- 不添加宽泛的 try/catch，错误必须显式传播
- 保持类型安全，避免强制类型断言
- 先搜索已有实现再决定是否新增

对于前端任务，还要特别注明：避免千篇一律的模板化设计，追求有辨识度的视觉表达。

常见误区：很多人在提示中写“代码要写得快、写得简洁”。但官方推荐的措辞恰恰相反——优先考虑正确性、清晰度和可靠性，而不是速度。把 Codex 当成“赶工的初级开发者”来用，效果反而不好。

对 Git 脏工作区的处理

这个细节很多人不会想到，但在多人协作或并行任务场景下特别重要——工作区可能包含其他人的未提交改动。提示中需要明确规定：

- 永远不要恢复不是自己做的改动
- 提交或编辑时，忽略与自己无关的变更
- 发现意外更改时立即停下询问用户
- 禁止使用 `git reset --hard` 等破坏性命令

三、工具配置：影响性能的关键环节

提示工程搞定了，接下来是工具配置。这部分的内容偏向实操，如果你的团队直接用 Codex CLI 或云端智能体，很多配置已经内置好了；但如果你通过 API 集成 Codex，这些细节会直接影响效果。

★ `apply_patch`：最重要的编辑工具

`apply_patch` 是 Codex 修改代码的核心工具，OpenAI 官方强烈建议使用标准实现，因为模型就是在这种 diff 格式上训练的。有两种接入方式：

- **Responses API 内置**：直接在工具列表中加入 `{"type": "apply_patch"}`，最简单的方式
- **自由格式工具**：使用 Lark 语法定义上下文无关文法，适合需要自定义行为的场景

两种方式输出的 diff 格式相同，模型都能正确使用。官方建议优先使用 Responses API 内置方式，因为它开箱即用且与模型训练时的格式完全一致；只有需要自定义解析逻辑或扩展行为时才考虑自由格式工具。

`shell_command`：字符串优于数组

一个容易忽视的细节：将命令作为单个字符串传递（而非字符串数组）效果更好。同时，工具描述中应要求“始终填写工作目录，避免在命令中使用 `cd`”，这能减少路径混淆。

并行工具调用

Codex 支持并行工具调用。通过设置 `parallel_tool_calls: true`，可以让模型同时发起多个工具调用，这比串行调用快不少。提示中应明确要求：

- 能并行的调用绝不串行
- 工作流程应该是：规划需要读取的资源 → 批量并行发出 → 分析结果 → 如有新的未知需求再重复

工具响应的截断策略

当工具返回的内容过长时，建议截断到约 10k Token（可用字节数除以 4 近似估算）。截断方式为：前半段保留开头内容，后半段保留结尾内容，中间用 `...N tokens truncated...` 格式的省略标记连接（其中 N 为截断的 Token 数）。这样既保留了关键上下文，又不会浪费 Token 预算。

工程提示：为什么要保留头尾两部分？因为工具输出的开头通常是摘要或状态信息，结尾往往是错误信息或最终结果——这两部分对模型决策最有价值。中间的重复性内容截断后影响最小。

四、AGENTS.md：项目级指令的分层机制

提示工程搞定了，接下来是另一个高频配置项——AGENTS.md。它的作用和 Claude Code 的 CLAUDE.md 类似，都是给 AI 注入项目级的上下文和规范。

★ 加载规则

Codex CLI 会自动扫描并注入 `AGENTS.md` 文件（也支持 `.codex` 等替代文件名），加载逻辑遵循分层覆盖原则：

1. 从用户主目录 `~/ .codex` 开始，沿仓库根目录到当前工作目录逐层扫描
2. 每个目录的指令独立成为一条用户消息
3. 子目录的指令会覆盖父目录的同名配置
4. 消息以根到叶的顺序注入对话历史

这意味着你可以实现分层配置：

层级	路径	适用范围
全局	~/ .codex/AGENTS.md	所有项目的通用默认行为（如语言偏好、通用编码风格）
项目	仓库根目录 AGENTS.md	项目级约定（如构建命令、测试规范、依赖管理）
模块	子目录 AGENTS.md	模块级特殊规则（如某个微服务的特定 API 约定）

实际示例：OpenAI 自己的 AGENTS.md

OpenAI 在 Codex CLI 的开源仓库中放置了一份真实的 AGENTS.md ，内容涵盖：

- Rust 代码风格约定（使用 `#[allow(clippy::xxx)]` 而非全局禁止 clippy 警告）
- TUI 界面的样式规则（使用 ratatui 框架）
- 测试策略（集成测试优先，单元测试为辅）
- API 开发规范（JSON 请求/响应格式、错误处理）

这份文件本身就是 AGENTS.md 最佳实践的参考范本。

五、安全模型：从建议到全自动

安全这一环不能跳过。Codex CLI 和云端智能体的安全机制差异较大，分开来说。

★ Codex CLI 的三级审批模式

Codex CLI 提供三种安全模式，对应不同级别的自动化需求：

模式	说明	适用场景
Suggest	可读取文件，但所有写操作和命令需确认	代码审查、学习
Auto Edit	自动编辑文件，但命令行操作需确认	日常开发
Full Auto	全自动，编辑和命令都自动执行	CI/CD、批量任务

在 Full Auto 模式下，Codex CLI 还提供沙箱机制来限制潜在风险：

- **macOS**：使用 Apple Seatbelt（`sandbox-exec`）将文件系统设为只读白名单，并完全阻断出站网络
- **Linux**：默认无沙箱，官方推荐使用 Docker 容器隔离，配合 `iptables` / `ipset` 防火墙脚本阻断除 OpenAI API 外的所有出站流量

拓展一下：Full Auto 模式下，Codex CLI 还会在非 Git 仓库中弹出一个警告确认，提醒你没有版本控制的安全网。这个设计细节挺贴心——在全自动模式下，Git 仓库的“可回滚性”是最后一道防线。

Codex 云端智能体的安全机制

云端智能体的安全设计更为严格：

- 每个任务在独立的容器中运行，完全没有网络访问权限
- 运行时间和资源消耗有明确限制

六、GPT-5.3 Codex API 的高级特性

本节内容适用于通过 Responses API 直接调用 `gpt-5.3-codex` 模型的开发者。Codex CLI 和云端智能体在内部封装了这些机制，用户无需手动配置。

上下文压缩

通过 Responses API 的 `/compact` 端点，Codex 可以压缩对话历史，使对话能够持续很多轮而不触碰上下文窗口限制。实际效果：

- 长时间任务不会因为上下文溢出而中断
- 超长任务链不再受典型窗口长度的限制
- Token 消耗比逐轮累积更可控

工程提示：`/compact` 端点是 ZDR（Zero Data Retention）兼容的，返回的是一个 `encrypted_content` 项。后续请求中直接传递这个压缩项即可，无需手动处理上下文摘要。这一点在官方文档中没有特别强调，但集成时必须注意。

★ Phase 机制

这是个容易踩坑的地方。GPT-5.3-Codex 引入了 `phase` 字段来区分模型输出的不同阶段：

- `null`：普通输出
- `commentary`：工作中对用户的进度更新
- `final_answer`：最终完成的交付

重要提示：`phase` 是 `gpt-5.3-codex` 的**必需项**（`required`），不是可选功能。如果不在历史消息中正确保留 `phase` 元数据，会导致显著的性能下降。此外，`phase` 字段只能附加在 `assistant` 消息上，不要添加到 `user` 消息中，否则会引发模型行为异常。

Preamble（进度更新）的节奏控制

Preamble 是模型在执行过程中向用户报告进度的机制。官方给出了明确的节奏建议：

- **目标频率：**每隔 1-3 个执行步骤发送一次进度更新
- **硬性下限：**至少每 6 个步骤或每 10 次工具调用必须发送一次
- 如果模型连续执行了大量操作而没有任何进度输出，用户会失去对任务状态的感知

这意味着在提示工程中，应当明确要求模型保持合理的进度汇报节奏，避免过于频繁（变成日志式更新）或过于稀疏（让用户失去上下文）。

两种协作个性

Codex 支持切换“友好”和“务实”两种个性风格：

风格	特点	适用场景
友好模式	更像热情的结对编程伙伴，确认多、解释细	新人引导、模糊需求探索、高风险改动
务实模式	简洁直接，每个 Token 的信息密度更高	延迟敏感、用户已熟悉 workflow

个性配置写在系统提示中，通过描述来引导模型的措辞风格、解释深度和热情程度。

推理强度选择

Codex 支持多级推理强度：

强度	说明	适用场景
medium	日常交互式编码推荐，在智能和速度之间取得平衡	大部分日常开发
high	较复杂的架构决策和重构任务	跨模块重构、复杂需求
xhigh	真正困难的多系统协调、复杂 bug 排查等场景	多服务联调、疑难 bug

选择合适的推理强度可以直接影响成本和响应速度。我的建议是：**先用 medium 跑，遇到明显推理不足的情况再升级，不要一上来就用 xhigh。**

七、常见问题与调试技巧

实际使用中，有几个高频问题值得单独拿出来讲。

★ 三个常见失败模式

OpenAI 官方追踪到了三个高频问题，每个都有对应的解法：

1. 过度思考

模型在执行第一次有用操作前耗时过长。解决方法是在提示中明确要求“立即开始行动”。

2. 日志式更新

模型机械地汇报状态而非自然协作。解决方法是在提示中要求“只在关键节点报告进度，避免机械式状态日志”。

3. 重复性口癖

反复使用“好发现”、“明白了”等填充词。解决方法是在提示中直接禁止这些表达。

工程提示：官方给出了一个很实用的调试技巧——“元提示”。做法是在模型的回复末尾追加反馈，要求它审视自己的指令并建议改进。生成几次回复后，取其共性的建议，就能得到有针对性的指令优化方案。本质上就是在让模型帮你写提示词。

自定义工具的调优

对于 Web 搜索、语义搜索、MCP 等非标准工具，模型没有专门的后训练，效果会打折扣。但可以通过以下方式弥补：

- 工具命名要精确（`semantic_search` 比 `search` 好）
- 在提示中明确说明何时、为何、如何使用每个工具，附带正反示例
- 让自定义工具的输出格式区别于模型已熟悉的工具输出，避免混淆

常见误区：很多人以为自定义工具只要定义好参数就行了。实际上，**工具的输出格式同样关键**——如果自定义工具的输出长得和 `ripgrep` 一模一样，模型可能会用错工具，因为它分不清两者的结果。让不同工具的输出在视觉上有明显区分，能有效减少混淆。

八、团队落地建议

最后聊几句团队层面的落地经验。

渐进式引入

建议团队按以下阶段逐步引入 Codex，不要一上来就 Full Auto：

1. **Suggest 模式试用：**让开发者熟悉 Codex 的代码理解能力和建议质量
2. **Auto Edit 模式日常使用：**在受控环境下逐步增加信任度
3. **Full Auto + 沙箱模式：**在 CI/CD 流水线或批量任务中启用全自动

AGENTS.md 的团队协作

为团队项目建立 AGENTS.md 时，建议覆盖以下内容：

- 项目构建和测试命令
- 代码风格和命名约定
- 依赖管理策略
- Git 工作流规范
- 常见陷阱和注意事项

成本控制

- 合理选择推理强度（medium 能覆盖大部分日常场景）
- 利用上下文压缩减少 Token 消耗
- 并行任务时注意监控总资源使用量

一句话：先用 **Suggest** 模式建立信任，再用 **Auto Edit** 提效，最后才考虑 **Full Auto**。
AGENTS.md 在团队推广前，最好先让一两个人试跑一周，把规则调顺了再全员铺开。

参考来源：

- OpenAI 官方博客：[Introducing Codex](#)
- OpenAI Codex CLI 开源仓库：github.com/openai/codex
- OpenAI 官方提示工程指南（中文译文参考）：liduos.com/posts/codex-prompting-guide
- OpenAI Codex 仓库 AGENTS.md 实际配置